CS5730 Operating Systems: 2019 Midterm Solutions

2019 · CS5730 Operating Systems Midterm (2019/12/12)

1. Short Answer: Your answers should be as concise as possible; aim for three or four sentences.

Q1.1 (5 pt) Ordinarily the `exec()` system call follows the `fork()`. Explain what would happen if a programmer were to inadvertently place the call to `exec()` before the call to `fork()`. 2019

ANSWER

exec() **replaces the calling process's entire memory image** (code, data, heap, stack) with the program named in its parameters. On success, **it never returns**.

So if exec() is placed before fork(), the new program starts running in place of the original one, under the **same process ID**. The rest of the original code, **including the fork() call, is gone and never executed**. **No child process is created**, and the original program's remaining work is not done.

Only if exec() **fails** (returns −1) does execution continue to the fork().

Q1.2 (5 pt) What happens if one thread of a multi-threaded process call `fork()`? What happens if one thread of a multi-threaded process call `exec()`? 2019

ANSWER

fork(): this depends on the system. Some UNIX systems provide **two versions** of fork(). One **duplicates all threads** of the process. The other (the POSIX behaviour) **duplicates only the thread that called fork()**, so the child is single-threaded. If exec() is called right after fork(), duplicating only the calling thread is enough, because exec replaces everything anyway. If the child does not call exec, duplicating all threads may be needed.

exec(): exec works as usual. The program named in its parameter **replaces the entire process, including all of its threads**. The other threads are destroyed, and the new program starts with a single thread.

Q1.3 (5 pt) What is processor affinity? Explain why it needed to be considered during process scheduling on multiple CPU systems. 2019

ANSWER

Processor affinity: a process has an affinity for the processor it is currently running on. It is kept on (or restricted to) that same CPU rather than migrated to another one.

Why it matters on multiple-CPU systems: while a process runs, its recently used data fills that processor's **cache**. If the scheduler migrates it to another CPU, the old cache contents must be **invalidated** and the new processor's cache must be **repopulated**. That causes many cache misses and memory stalls, which is expensive. Keeping the process on the same CPU reuses the "warm" cache. On NUMA systems it also keeps the process close to its local memory.

Soft affinity: the OS tries to keep the process on the same CPU but may migrate it. **Hard affinity:** the process specifies the set of CPUs it may run on. The scheduler must balance affinity against **load balancing** (push/pull migration), which moves processes from busy to idle CPUs.

Q2 (10 pt) One technique for implementing lottery scheduling works by assigning processes lottery tickets, which are used for allocating CPU time. Whenever a scheduling decision has to be made, a lottery ticket is chosen at random, and the process holding that ticket gets the CPU. The BTV operating system implements lottery scheduling by holding a lottery 50 times each second, with each lottery winner getting 20 milliseconds of CPU time (20 milliseconds × 50 = 1 second). Describe how the BTV scheduler can ensure that higher-priority threads receive more attention from the CPU than lower-priority threads. 2019

ANSWER

Give **higher-priority threads more lottery tickets**. At each of the 50 drawings per second, the winning ticket is chosen uniformly at random. A thread holding k of the T tickets therefore wins with probability k/T, and on average receives about **k/T of the CPU time** (k/T × 50 × 20 ms per second).

Example: high-priority thread H has 30 tickets and low-priority L has 10. H wins about 75% of the drawings (≈ 750 ms per second) and L about 25% (≈ 250 ms). The scheduler can also adjust priority dynamically by giving or taking away tickets, e.g. extra tickets to a thread that has waited long. Every thread with at least one ticket eventually wins, so there is no starvation.

Q3 (15 pt) A variation of the round-robin scheduler is the **regressive round-robin scheduler**. This scheduler assigns each process a time quantum and a priority. The initial value of a time quantum is 50 milliseconds. However, every time a process has been allocated the CPU and uses its entire time quantum (does not block for I/O), 10 milliseconds is added to its time quantum, and its priority level is boosted. (The time quantum for a process can be increased to a maximum of 100 milliseconds.) When a process blocks before using its entire time quantum, its time quantum is reduced by 5 milliseconds, but its priority remains the same. What type of process, namely CPU-bound (such as batch processes) or I/O-bound (such as interactive processes), does the regressive round-robin scheduler favor? Explain. 2019

ANSWER

It favors CPU-bound processes.

- A process that **uses its entire quantum** (does not block for I/O) is CPU-bound behaviour. It is **rewarded**: +10 ms of quantum, up to 100 ms, **and** a priority boost.
- A process that **blocks before using its quantum** is I/O-bound / interactive behaviour. It is **penalized**: −5 ms of quantum, and its priority is **not** raised.

So over time CPU-bound (batch) processes get longer time slices and higher priority, and I/O-bound processes get shorter slices and stay at lower priority. This is the opposite of the usual goal of favoring interactive processes.

Q4 (10 pt) Consider the following code segment: 2019

```
pid_t pid;

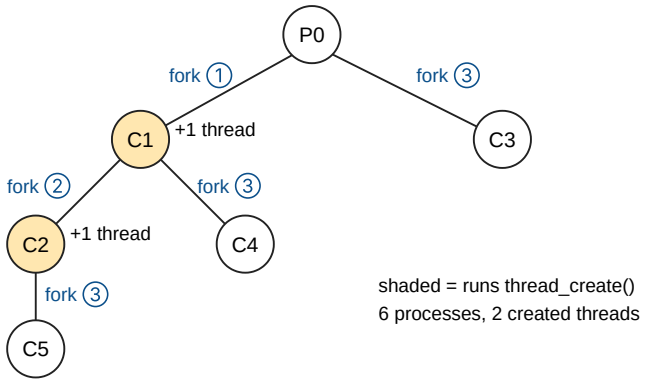
pid = fork();
if (pid == 0) { /* child process */
    fork();
    thread_create( . . .);
}
fork();
```

a) How many unique processes are created?
b) How many unique threads are created?

ANSWER

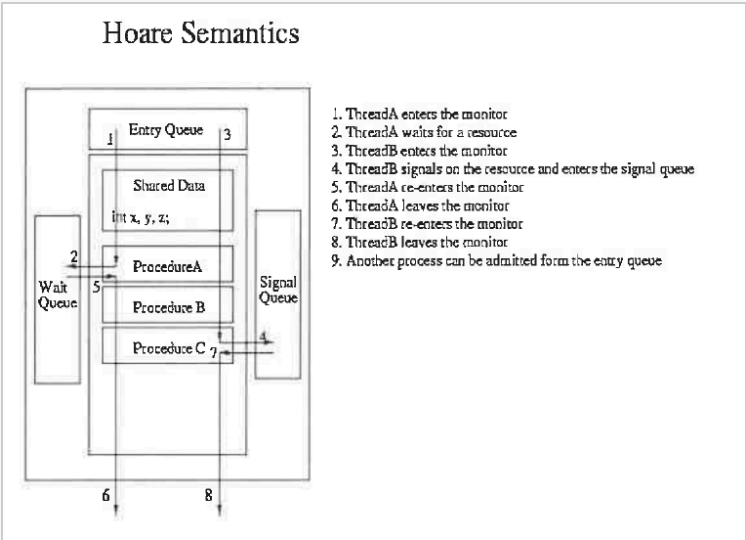
Trace each fork (the original process is P0):

- fork ① gives P0 and child C1. Only C1 has pid == 0.
- Inside the if, C1 calls fork ②, creating C2. **C1 and C2 each call thread_create()**, so 2 threads are created.
- The final fork ③ is executed by P0, C1 and C2, creating C3, C4 and C5.



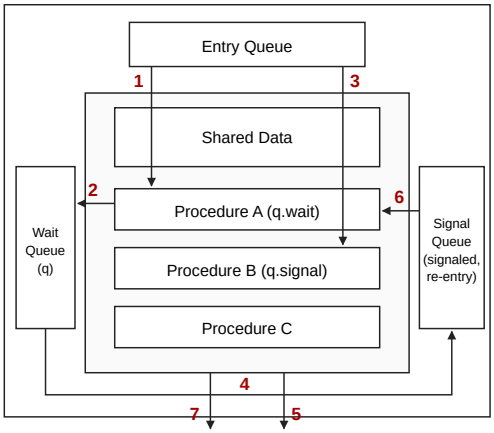
(a) 6 unique processes (P0, C1–C5; 5 newly created if the original is not counted).
(b) 2 unique threads created by thread_create(), one in C1 and one in C2. The last fork() duplicates only the calling thread (POSIX), so it creates no extra threads.

Q5 (10 pt) Note that only one process is active within the monitor. Suppose `A` associated with the condition variable `q` is resumed when `B` invokes `q.signal()`. We discussed two possibile solutions in class. One is **signal and wait**, and the flow is illustrated in the following (Hoare semantics). Please modify the Hoare semantics and depict the other solution (i.e., **signal and continue**). 2019



ANSWER

Signal-and-continue: when B executes `q.signal()`, **B keeps running** in the monitor. The waiting process A is only moved from q's wait queue to a ready/re-entry queue (the "signal queue" now holds the *signaled* process). A resumes after B leaves the monitor or waits.



1. Thread A enters the monitor
2. A waits on q → q's wait queue (monitor released)
3. Thread B enters the monitor
4. B signals q: A is moved from wait queue → signal queue; **B CONTINUES running**
5. B leaves the monitor
6. A re-enters the monitor (signal queue has priority over entry queue)
7. A leaves the monitor
8. Another process can be admitted from the entry queue

Difference from Hoare: signal queue now holds the SIGNALED process, not the signaler; 8 steps instead of 9.

Compared with Hoare: in Hoare's version A runs immediately and the signaler B waits in the signal queue. In signal-and-continue, A may find the condition changed by the time it runs, so waiters should re-check it with `while (!cond) q.wait();`. There are fewer context switches, and this is what Java and Pthreads use.

Q6 (20 pt) In class, we have illustrated the solution for dining-philosopher problem using monitor. However, as I mentioned, the solution is conceptual only since there is no monitor mechanism. We also discussed how to implement a monitor mechanism with **signal and wait** via semaphores, and thus the dining-philosopher problem solution together with the monitor mechanism with **signal and wait** via semaphores is a complete solution. Please modify the monitor mechanism with **signal and wait** via semaphores to **signal and continue** and integrate them into dining-philosopher problem solution. That is, I would like you to present a complete code for dining-philosopher problem solution which uses a monitor mechanism with **signal and continue** via semaphores. 2019

ANSWER

Combine the dining-philosopher monitor with the signal-and-continue implementation above. `self[i]` becomes semaphore `self_sem[i]` plus counter `self_count[i]`.

```
enum {THINKING, HUNGRY, EATING} state[5];
semaphore mutex = 1;            // monitor lock
semaphore next  = 0;      // signaled, waiting to re-enter
int next_count  = 0;
semaphore self_sem[5]   = {0,0,0,0,0};  // cond. self[i]
int       self_count[5] = {0,0,0,0,0};

void leave() {                // release the monitor
    if (next_count > 0) signal(next);
    else                signal(mutex);
}
void self_wait(int i) {          // self[i].wait()
    self_count[i]++;
    leave();                  // release monitor
    wait(self_sem[i]);           // sleep until signaled
    wait(next);               // re-enter after signaler
    next_count--;
}
void self_signal(int i) {     // self[i].signal(), S&C
    if (self_count[i] > 0) {
        self_count[i]--;
        next_count++;
        signal(self_sem[i]);  // wake i; caller continues
    }
}
void test(int i) {            // only inside the monitor
    if (state[(i+4)%5] != EATING && state[i] == HUNGRY &&
        state[(i+1)%5] != EATING) {
        state[i] = EATING;
        self_signal(i);
    }
}
void pickup(int i) {
    wait(mutex);                  // enter monitor
    state[i] = HUNGRY;
    test(i);
    while (state[i] != EATING)  // re-check after waking
        self_wait(i);
    leave();
}
void putdown(int i) {
    wait(mutex);                  // enter monitor
    state[i] = THINKING;
    test((i+4)%5);            // left neighbor may eat
    test((i+1)%5);            // right neighbor may eat
    leave();
}
void init() {
    for (int i = 0; i < 5; i++) state[i] = THINKING;
}

void philosopher(int i) {
    while (true) {
        think(); pickup(i); eat(); putdown(i);
    }
}
```

It is correct under signal-and-continue because test() sets state[i] = EATING **before** signaling, and only philosopher i can change it back. So when i resumes, its condition still holds. A philosopher eats only when neither neighbor eats, so there is no deadlock (starvation is still possible).

7. Multilevel queue scheduling

Q7.1 (5 pt) List two possible scheduling among queues, which we discussed in class. 2019

ANSWER

Two ways to schedule **among** the queues of a multilevel queue:

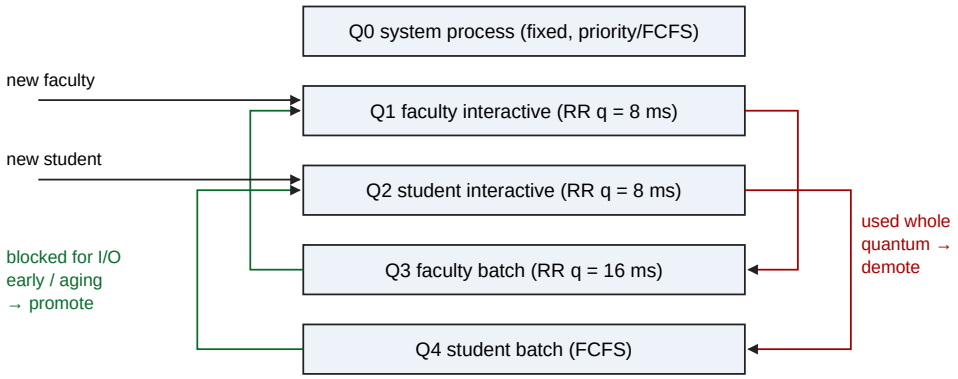
1. **Fixed-priority (preemptive) scheduling.** Serve all processes in a higher-priority queue (e.g. foreground/interactive) before any process in a lower queue (background/batch). A process arriving in a higher queue preempts a lower one. Lower queues may **starve**.
2. **Time slicing among queues.** Each queue gets a fixed share of CPU time, which it schedules among its own processes. Example: 80% to the foreground queue (RR) and 20% to the background queue (FCFS).

Q7.2 (5 pt) Modify the following multilevel queue scheduling system such that a new process will automatically enter the appropriate queue after a few runs. 2019

system process
faculty interactive process
student interactive process
faculty batch process
student batch process

ANSWER

Turn it into a **multilevel feedback queue**: a process may **move between queues** according to its observed behaviour, so it reaches the right queue automatically after a few runs.



Higher queue preempts lower (fixed priority between queues); Q0 never moves.

- A new process enters the **interactive** queue of its class (faculty → Q1, student → Q2), which uses a short quantum.
- If it **uses its whole quantum**, it is CPU-bound, so it is **demoted** to the batch queue of its class (Q1→Q3, Q2→Q4).
- If it **blocks for I/O before the quantum ends**, it is interactive, so it stays in or is **promoted** back to the interactive queue (Q3→Q1, Q4→Q2).
- **Aging** promotes processes that wait too long, preventing starvation. System processes stay fixed in Q0.

Defined by: number of queues, scheduling algorithm per queue, when to promote, when to demote, and which queue a new process enters.

Q8 (10 pt) A student is asked to write a program which will create a hundred POSIX threads. Each thread will print out the value of the parameter passed from `main()` which are 1, 2, 3, up to 100 respectively. Consider the following code fragment he wrote, please verify its correctness. If incorrect, please correct the error. 2019

```
void *doSomething(void *y) {
    ...
    printf("%d\n", *y);
    ...
}

int main() {
    pthread_t t1[100];          /* declare a thread */
    int param1, i;

    param1 = 1;
    for (i = 0; i < 100; i++) {
        param1++;
        pthread_create(&(t1[i]), NULL, doSomething, &param1);
    }
    ...
}
```

ANSWER

Incorrect. Problems:

1. **All 100 threads receive the same address** `&param1`. main keeps incrementing param1 while the threads run, so each thread prints whatever value param1 holds when that thread happens to run. The output has duplicates and missing numbers (often many 100s), not 1...100.
2. main never calls **pthread_join**, so it may return (ending the process and all its threads) before the threads print.
3. `*y` dereferences a `void *`, so it needs a cast to `int *`.

Corrected: give every thread its own argument and join all threads.

```
void *doSomething(void *y) {
    ...
    printf("%d\n", *(int *)y);
    ...
    pthread_exit(0);
}

int main() {
    pthread_t t1[100];            /* declare threads */
    int param[100], i;      /* one argument per thread */

    for (i = 0; i < 100; i++) {
        param[i] = i + 1;          /* 1, 2, ..., 100 */
        pthread_create(&(t1[i]), NULL, doSomething, &param[i]);
    }
    for (i = 0; i < 100; i++)
        pthread_join(t1[i], NULL);  /* wait for all */
    return 0;
}
```

(If the numbers must also be printed *in order*, the threads must additionally be synchronized, e.g. by joining each thread right after creating it.)